

The Pathfinder engine

Pathfinder, github.com/vd1/pathfinder

12 September 2026, revised 13 September

Abstract

1 Introduction

The pathfinder program. Pathfinder takes two corpora of scientific papers (concretely arxiv deposits), Q and P , and, using agentic methods, looks for pairs (q, p) in $Q \times P$ with enough of an interesting connexion to try and elaborate it as a pre-publication. Ideally, the pre-publication satisfies the usual constraints of correctness, novelty, relevance, readability, and exposition of related work in the literature.

In some cases, the input tuple may be supplemented with a seed note (provided by the agent that scanned and scored the pair in the first place) that is a paragraph describing the putative connexion.

This type of task is commonly carried out by human experts. It happens through seminars that the seed of an idea is provided by someone in the audience, and such connexions are made, and, in favourable circumstances, further pursued all the way to a publication. However, by using modern agents (which are competent experts on every corpus, and always “willing” to contribute), one can do it faster, cheaper, and, therefore, at a larger scale. We have ran it on sets of paper of size about a hundred each for a reasonable token budget.

[small epistemologic paragraph on creativity vs proving conjectures; and whether creativity is by connexion or otherwise]

Object of this note. This note purports to describe in implementable terms a simple agentic process whereby an input tuple is progressively elaborated and in the favourable cases a pre-publication is generated.

As we will see below, the process recruits various agents in various roles: researchers, verifiers, editors, etc. It consists broadly in two phases.

1. The research phase: researcher agents collaborate and produce a draft paper via rounds of interactions with a verifier agent. This is the creative part of the process which can end on paused thread, or a draft. In fact, running this phase again may lead to a different outcome, possibly a draft expounding on a different idea. (The next paragraph gives an example.)
2. The editing phase: an author agent produces via rounds of interactions with a reviewer agent, a final version of the draft. The final draft can put on pause or accepted as a pre-publication by the reviewer.

An example, and why reruns differ. Consider the following (q, p, c) input triplet:

$q = \text{arXiv:2608.29130}$ “A Systematic Approach to Mechanism Design with Stochastic Dynamic Stability”, which designs quadratic payments so that self-interested agents settle at the welfare-maximising allocation and learns that equilibrium from sampled feedback

$p = \text{arXiv:2609.04460}$ “Distributed risk-averse optimization via CVaR”, which minimises conditional value-at-risk over a network from noisy function values.

The seed c written by the scanning agent, is: replace q ’s expected-value objectives with CVaR and use p ’s gradient estimator inside q ’s learning loop; does the payment mechanism still work?

The thread was run twice on (q, p, c) . Both runs quickly rejected the literal splice (q needs a best response, p supplies a gradient step) and both ended as DRAFT, but with different results. The first run proved a positive statement: a uniform bound $\sup_x |\tilde{C}_n(x) - C_n(x)| \leq L_n \delta_n + (U_n/\alpha_n) \sqrt{2\pi/s_n}$ between p ’s smoothed, sampled surrogate and the true CVaR objective, and drew from it welfare and individual-rationality guarantees for the risk-averse mechanism. The second run proved obstructions instead: a two-point example in which the expected objective is strongly concave but its CVaR is constant, so the curvature q ’s uniqueness proof needs can vanish under the substitution, and a coupling example showing that the local risk oracles both papers use cannot determine system-wide tail risk. Each note is correct, each was accepted by a verifier that had not taken part, and a referee would want both in one paper.

This is the general situation. The research phase is a sampler: the same pair and seed, rerun, draws a different subset of what the pair supports, and the spread between reruns is at least as large as the effect of any setting we varied. For a pair that drafts, a second independent run and a consolidation of the two notes is therefore worth more than a longer single run, and cheaper than a third researcher on every thread.

The various stages of the process The process produces the following sequence of file outputs:

1. a conversation *thread* recorded on a ledger of everything the researcher agents said to each other
2. a *note* consigning to paper form what the researcher agents and verifier agreed on (together with a *readable note* of the same, rewritten for a reader who never saw the ledger), and,
3. when accepted by the reviewer, a *paper*

The outputs above are arranged in increasing order of readability by humans, and decreasing dependence on the history of the generation process.

The readable note presents the research phase’s final outcome for expert assessment, including unsuccessful or inconclusive outcomes.

Why a state machine. The work is done by model calls that cost money, take minutes, fail, and disagree. Writing the pipeline as a machine with explicit states, transitions and budgets is what makes it re-implementable by a colleague on another corpus, auditable after the fact (every decision traces to a file and a receipt), and safe to interrupt: a stop, a crash or an exhausted budget leaves the machine in a named state from which it resumes, and a human can see at a glance where each pair stands and what would move it.

How. One research thread is one instance of the machine described below. Its subject is a pair of papers together with the seed of an idea: in Pathfinder the seed is the scan’s connexion, rationale and scores, handed to the peers in their first prompt; in the agQSL instance of the same machine

the seed is a written idea artefact. The machine does not care where the seed came from, only that the thread starts from a pair and a seed. The campaign is a set of these instances plus a scheduler with a purse. Three kinds of actor appear: *agents*, model calls with a prompt and a bounded view of the state, of which the two *judges* (the verifier of a note, the reviewer of a paper) make the only decisions; the *pipeline*, code that builds, checks, records and moves the position; and the *owner*, whoever types in the terminal, a human or a delegated session, who raises budgets, edits by hand and settles what is paused. The agents of one thread share one append-only ledger; that ledger, the files beside it and a position record are the whole state of the thread. Everything below is what the code does; the builder prompt points here for the shape to re-implement.

2 Global state of a thread

Position. `stage`, where the machine is; `status`, one of `running`, `stopped`, `BLOCKED` or a terminal word; the counters `round` (research rounds used), `repairs` (note repairs used), the paper's `round` and the editor's `attempt`.

Memory, all files under `threads/<pair>/`:

- `inputs/`: the two papers, flattened TeX or PDF text, and their metadata; the seed, the pair's row of `scan.jsonl`, is read from the campaign level.
- `ledger.jsonl`: append-only attributed entries of kind idea, finding, objection, correction, intention, ready or review; the peers' shared memory and the verifier's evidence.
- one directory per peer (`ada/`, `emmy/`, ...): that peer's derivations, scripts and checks. The peers are a campaign setting (`peers`); two by default, three were tested.
- `<pair>.tex`: the consolidated note; `<pair>.verdict.json`: every verdict with the digest of the note it judged.
- `paper/`: the paper, its bibliography, its search record, the pipeline's checks per round, and every review with the digest of the paper it judged.
- `edited/`: the readable note (`note.tex`, `references.bib`, `note.pdf`).
- `status.json`, `paper/paper.json`, `edited/edit.json`: the position of each stage; `lock`: the pid of the runner working the thread.

Campaign level. `fetch.json` (the two queries and how far each side has been paged), `Q.jsonl` and `P.jsonl`, `scan.jsonl`, `shortlist.json`, `receipts.jsonl` (the only spend figure), and the markers `stop.json` and `health.json`.

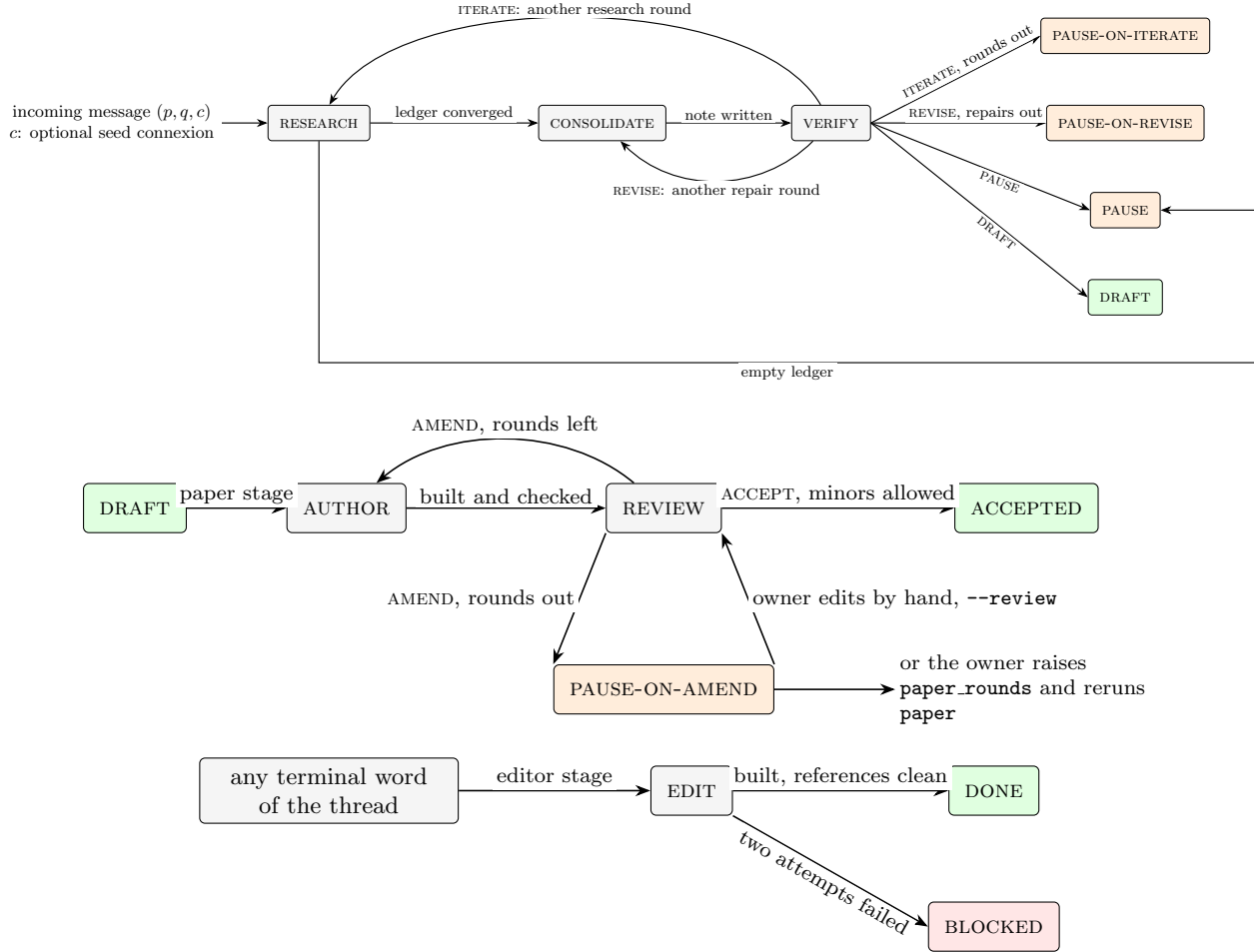
3 The ledger

The ledger is the one piece of state several agents write. Its rules are what make the peers a group rather than a set of monologues:

- Entries are appended through a helper, never edited; each carries a sequence number, an actor, a kind and a time. Peers read the whole ledger before every call.
- *ready* is a claim about the ledger as it stands: it is void as soon as anyone adds a substantive entry (anything but ready or review) after it. A peer whose ready still stands is not called again in that round; it waits for its partners.
- The round ends when every peer's ready stands, or when the round's allowance is out.

- The verifier writes its verdict into the ledger as a review entry, so the next round’s peers, and the consolidator on a repair, read the judge’s reasons where they read everything else.

4 States



Terminal words of the research thread: DRAFT, PAUSE, PAUSE-ON-ITERATE, PAUSE-ON-REVISE. Terminal words of the paper: ACCEPTED, PAUSE-ON-AMEND. Endings of the editor: DONE, BLOCKED. Every PAUSE-ON-X means “the loop named X ran out of its budget with the judge still asking”; the owner can raise the budget and resume from the recorded stage, or, for the paper, edit it by hand and ask for one more review. The editor runs once for every terminal thread, whatever its ending, so that even a paused pair leaves a readable account.

stopped and **BLOCKED** are not endings. **stopped** is a drain checkpoint (a stop marker, a transport failure, a kill) and resumes at the recorded stage on the next start. **BLOCKED** is an unreadable decision or a missing output: the reply is saved beside the position (for the verifier, `verify-unreadable.txt`), the runner never re-admits the pair, and **reconcile** names the one safe action.

stage	decisions of its judge	endings
research thread	DRAFT, REVISE, ITERATE, PAUSE	DRAFT, PAUSE, PAUSE-ON-ITERATE, PAUSE-ON-REVISE
paper	ACCEPT, AMEND	ACCEPTED, PAUSE-ON-AMEND, blocked
editor	none	DONE, blocked

5 Transitions

Each transition is one or more model calls, or a mechanical step by the pipeline or the owner. “Sees” is the part of the global state the agent can read; “writes” is what it may change. Nothing else is visible or writable, which is what makes the record auditable.

Research thread

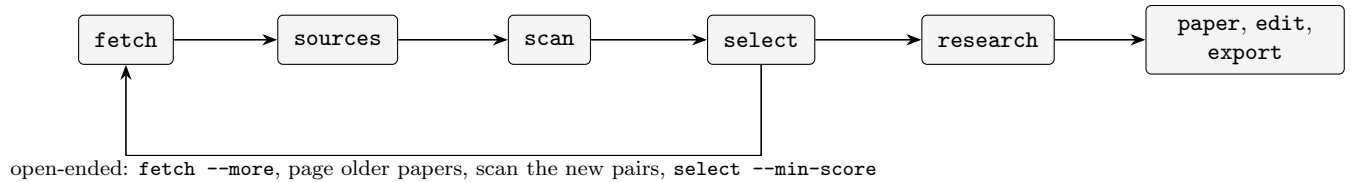
from → to	agent, prompt, tools	sees	writes	budget
corpus → SCANNED	judge, scan.md , no tools	Q and P titles and abstracts, or full text	one row of scan.jsonl	one call, retried once if unparseable
SCANNED → NEW	none (select)	all scan rows	shortlist.json : the top cut, or every pair at or above a threshold; started pairs are kept	none
NEW → PEERS	none (runner)	shortlist, statuses, locks, receipts, markers	lock, status.json	seats; guard: spend plus in-flight estimate under budget.usd , else the stop marker
PEERS → PEERS	every peer concurrently, peer.md , tools and web search	inputs, ledger, own and partners’ directories, the scan’s connexion and scores, this call’s allowance	ledger entries through the helper, own directory	peer_calls per peer and peer_seconds shared, per round; a peer whose ready stands waits instead of calling
PEERS → CONSOLIDATE	none	ledger readiness	status.json	when every peer is ready or the allowance is out; an empty ledger goes to PAUSE
CONSOLIDATE → VERIFY	first peer, consolidate.md , tools	ledger, all directories, the prior note; on a repair also the verifier’s corrections	<pair>.tex ; after an ITERATE it appends to the prior note, on a repair it rewrites	consolidate_seconds ; rerun once unless the note exists
VERIFY → next	verifier, verify.md , no tools	inputs, ledger, note, all inline	verdict file with the note’s digest, a review entry in the ledger, status.json	verify_seconds ; rerun once on an empty reply; an unreadable verdict is saved and the pair is BLOCKED
VERIFY → PEERS	verdict ITERATE		round +1	rounds (4); at the cap PAUSE-ON-ITERATE
VERIFY → CONSOLIDATE	verdict REVISE	the consolidator also sees the corrections	repairs +1	repairs (1); at the cap PAUSE-ON-REVISE

Paper, owner and editor

from → to	agent, prompt, tools	sees	writes	budget
DRAFT → REVIEW	author, author.md , tools and web search	everything in the thread, the web, the previous review's findings if any	paper/: paper.tex , references.bib , search.md	paper_seconds per call
REVIEW → next	reviewer, review.md , no tools	paper, bib, the pipeline's build and reference checks, the search record, note, ledger, inputs, all inline	paper/review.json with the paper's digest, paper.json	review_seconds ; paper_rounds (3); at the cap PAUSE-ON-AMEND. ACCEPT may carry minor findings; AMEND only for a finding that changes a claim, its scope, attribution or correctness
PAUSE-ON-AMEND → REVIEW	the owner edits the paper by hand, then paper PAIR --review	the paper as it stands	one more review, counted as the next round	none: an owner round is outside the budget
PAUSE-ON-X → its loop	the owner raises the budget and reruns research or paper	the position	the loop continues from the recorded stage	the raised cap
terminal → EDIT	editor, editor.md , tools, no search	inputs and metadata, note, verdicts, ledger, all directories; on retry, the build log and the reference findings	edited/	edit_seconds ; two attempts, then blocked

Between transitions the pipeline itself, with no model, does the checks that gate the next state: it builds the $\text{\LaTeX}$, verifies that every citation exists and is used and that arXiv identifiers carry their titles, refuses an edited note whose references do not check, records digests, and writes the position. The verifier's and reviewer's judgements are the only decisions; every other arrow is mechanical or the owner's.

6 The campaign



scan.jsonl is append-only, so a longer corpus only adds rows; a threshold cut re-selects over all rows and keeps every pair that has a thread. **explore** runs the loop **fetch --more**, **scan**, **select --min-score** for a number of passes. The runner then admits whatever is NEW on the shortlist, so research can run while the corpus grows.

7 Budgets, and what exhausting them means

budget	scope	when exhausted
peer_calls, peer_seconds	one peer round	consolidate on the ledger as it stands
rounds	the ITERATE loop	PAUSE-ON-ITERATE
repairs	the REVISE loop	PAUSE-ON-REVISE
paper_rounds	the AMEND loop	PAUSE-ON-AMEND
editor attempts	the build and reference checks	blocked
budget_usd	the campaign, at admission	stop marker; threads in flight finish; NEW pairs wait

Every model call appends a receipt, so the campaign budget is checked against the sum of receipts, never against an estimate alone. A call that times out with no usage is charged at `call_estimate_usd`. Owner rounds are not budgeted: the owner is the budget.

8 Interrupts and hazards

- A stop marker (operator, guard, or first Ctrl-C) stops admission; calls in flight land and write their checkpoints; the runner exits; the next start resumes each pair at its recorded stage.
- No session within the grace time is a transport failure: no receipt, the pair is **stopped**, two in a row set the health marker and pause admission until a probe succeeds. The **paper** and **edit** commands skip the failed pair and continue.
- **reconcile** reads the position of one pair and names the one safe action: start, resume peers, run consolidate, run verify, or nothing; **--apply** performs it.
- The lock protects a thread from two runners. It does not protect the paper or editor stage from two concurrent commands on the same pair; that is an operator error the code does not yet catch.

A Agent resources against human resources, per pair

Agent figures are taken from the receipts of the reference campaign (Claude Opus peers, Claude Sonnet scan; 14 threads, 5 papers), as medians over threads. Means are dragged up by three threads that hit time-outs and by one paper whose seven review rounds were spread over two days. Human figures are estimates for a postdoc-level researcher at a fully loaded cost of about 60 USD per hour, stated so that they can be rescaled.

agent, per pair	calls	USD	wall time	notes
scan	1	0.03	11 s	100 pairs for 3.34 USD
research (peers, consolidate, verify)	12	6.8	8 min	mean 9.8 USD and 51 min; worst thread 30 USD and 6 h
paper (author, review)	4 to 11	3.5	5 min per round	5 pairs; the worst took 14.7 USD over 7 rounds
readable note (editor)	1	1.0	2 min	
pair to reviewed paper	about 20	about 12	about 20 min	about 8 USD and 10 min for a pair that pauses

A second run of eight pairs with three peers on the Codex model gave a research stage of 7.8 USD and 7 minutes per pair: the model choice moves the cost little, the peer count moves the time.

human, per pair, same deliverables	hours	USD	calendar
scan two abstracts and decide	0.25	15	minutes
read both papers, derive, write the note	25 to 40	1,500 to 2,400	3 to 5 days
a colleague verifies the note	2 to 4	120 to 240	when free
paper with literature search	60 to 80	3,600 to 4,800	2 weeks
a referee's review	4 to 8	240 to 480	weeks of latency
readable write-up for outsiders	4	240	half a day
pair to reviewed paper	about 100	about 6,000	3 to 4 weeks

Per pair the agent pipeline is therefore roughly 500 times cheaper and 100 times faster on the calendar. At the scale of a campaign the gap is what makes the method exist at all: scanning 100 pairs costs 3 USD against 25 human hours, and the 14 threads and 5 papers of the reference campaign cost 175 USD, about three loaded postdoc days, against something like a working year.

Three caveats keep the comparison fair.

- *Not the same quality.* The accepted papers are correct technical remarks, the kind a good student writes in a week, not the pair a senior researcher would choose nor the paper they would write. A human would also never look at 100 pairs; taste replaces breadth.
- *Human time does not go to zero.* The owner still reads the readable notes, clears the decision queue and settles the paused pairs: on this campaign of the order of 10 to 20 minutes per pair, plus an hour of attention and about 12 USD for the one paper that was rescued by hand. The pipeline moves human effort from producing to choosing and reading.
- *Wall time is set by the slowest call, not by the sum.* Peers run concurrently, so a thread's clock is its longest call and its time-outs; the 8-minute median hides a 6-hour worst case.